

MAX GUI 智能体系统架构报告

版本：v1.1.0

更新日期：2026-08-07

代码基线：当前工作区（含未提交的文档更新）

1. 报告定位

本报告说明 MAX 当前实现基线、技术选型、目标能力与总体架构。报告把“当前已经可以运行的基础设施”和“后续要实现的 GUI 智能体闭环”分开描述，避免将设计中的模块误认为已经交付。

当前仓库的可运行基线是一个 Windows 本地原型：通过 CLI 启动 Doctor、模型管理和单图离线基准，也可以启动一个尚未接入模型后端的聊天控制台。当前系统不会默认操作真实桌面，不会隐式下载模型，也不会把模型身份写入 Doctor 证据。

目标系统是在此基础上逐步实现以下闭环：

用户目标 → 屏幕感知 → 任务规划 → 安全审批 → 桌面执行 → 结果验证 → 重试/恢复 → 审计归档

2. 设计目标与边界

2.1 目标

- 面向授权的 Windows 本地测试环境，优先支持单用户、单进程、单模型和单并发推理。
- 将观察、动作、审批、执行回执和验证结果结构化，保证每一步可追踪、可解释、可复核。
- 通过显式的安全 Guard 限制窗口、进程、坐标、动作频率、会话能力和人工确认。
- 使用本地模型和本地证据目录建立可重复的实验闭环。
- 先交付稳定的工具契约，再逐步替换模型、OCR 或桌面控制实现。

2.2 非目标

当前阶段暂不引入：

- 面向真实业务软件的无约束自动化；
- 桌面前端、浏览器扩展、长期记忆和多 Agent 协作；
- 在线训练、向量数据库、常驻网络服务、vLLM 推理服务；
- PostgreSQL、Redis、消息队列等跨进程基础设施。

3. 技术栈：用什么实现什么

技术/组件	计划承担的职责	当前状态
Windows 11	目标桌面、窗口、DPI 和输入环境	已实现： Doctor 支持基础环境核验和显式桌面探针

技术/组件	计划承担的职责	当前状态
Python 3.12 + venv/uv/Conda	统一运行时版本与可替换的环境隔离方式	已实现 : 根 <code>requirements.txt</code> 提供统一安装入口; 当前核验使用 Conda <code>max</code>
Typer + Rich + Prompt Toolkit	首要 CLI 命令入口、终端渲染和默认交互式会话	已实现 : 基础交互、 <code>/doctor</code> 、 <code>/quit</code> 和未配置后端提示
Textual	后续可选的高级 UI 和状态展示入口	部分实现 : 已有控制台壳; 需显式选择, 尚未连接 Agent 后端
LangGraph	编排器的显式状态图、循环、预算和终止条件	仅依赖可用 : 版本已锁定, Orchestrator 尚未实现
LangChain	提示词、模型与工具的适配层	仅依赖可用 : 版本已锁定, 尚未接入模型工具; 不承载高风险执行逻辑
PyTorch CUDA 13.0	GPU 推理和 BF16 张量运算	已实现 : CUDA/BF16 已验证; 版本与 wheel 索引由根 <code>requirements.txt</code> 约束
Transformers	本地多模态模型加载、Processor 和单图推理	已实现 : 离线 Qwen3.5-4B 加载路径和基准接口
ModelScope	模型元数据校验和显式权重下载	已实现 : 下载目标必须位于仓库 <code>model/</code> 目录
<code>mss</code>	屏幕捕获	部分实现 : 已接入 Doctor; Perception 工具尚未实现
OpenCV + NumPy	图像转换、裁剪、差分和基础视觉处理	部分实现 : 已接入 Doctor 合成图检查; Perception 工具尚未实现
PaddleOCR	屏幕文字识别	仅依赖可用 : 当前只检查包版本; OCR 工具尚未进入 Agent 闭环
UI Automation / <code>pywinauto</code>	获取标准 Windows 控件树和窗口属性	仅依赖可用 : 当前环境可导入, 尚未形成工具契约
PyAutoGUI / <code>pynput</code>	受 Guard 审批后的鼠标、键盘和滚动输入	部分实现 : 仅在显式桌面探针中验证基础输入; Guard 与 Controller 尚未实现
Pydantic / Protocol	数据契约校验、序列化和模型 Provider 接口	部分实现 : <code>ModelProvider</code> Protocol 已定义; 完整 Pydantic schema 待实现
JSONL Artifact Store	运行配置、环境、轨迹和结果归档	已实现 : 位于 <code>artifacts/</code> , 适合单用户单进程审计
<code>unittest</code> + <code>pip check</code>	单元测试和依赖一致性验证	已实现 : 当前 38 项测试通过, 依赖检查通过

当前状态统一分为四类：**已实现**表示代码和命令已存在且完成当前核验；**部分实现**表示已有基础接口或辅助能力，但尚未进入端到端闭环；**仅依赖可用**表示运行环境已具备第三方组件，项目工具契约尚未实现；**规划中**表示仅属于目标设计。

3.1 关键选型理由

第一版采用“Python 单进程 + LangGraph 编排 + 本地 PyTorch/Transformers 推理 + 工具契约”的组合。这样可以减少服务拆分、HTTP 序列化和跨进程状态带来的不确定性，同时保留替换感知、模型和执行器的接口边界。

LangGraph 只负责显式状态图，不把高风险动作藏在自由循环或通用 Agent Executor 中。LangChain 只承担提示词和模型/工具适配。真正的桌面输入必须经过独立 Guard，模型输出只能产生候选动作，不能直接获得鼠标键盘权限。

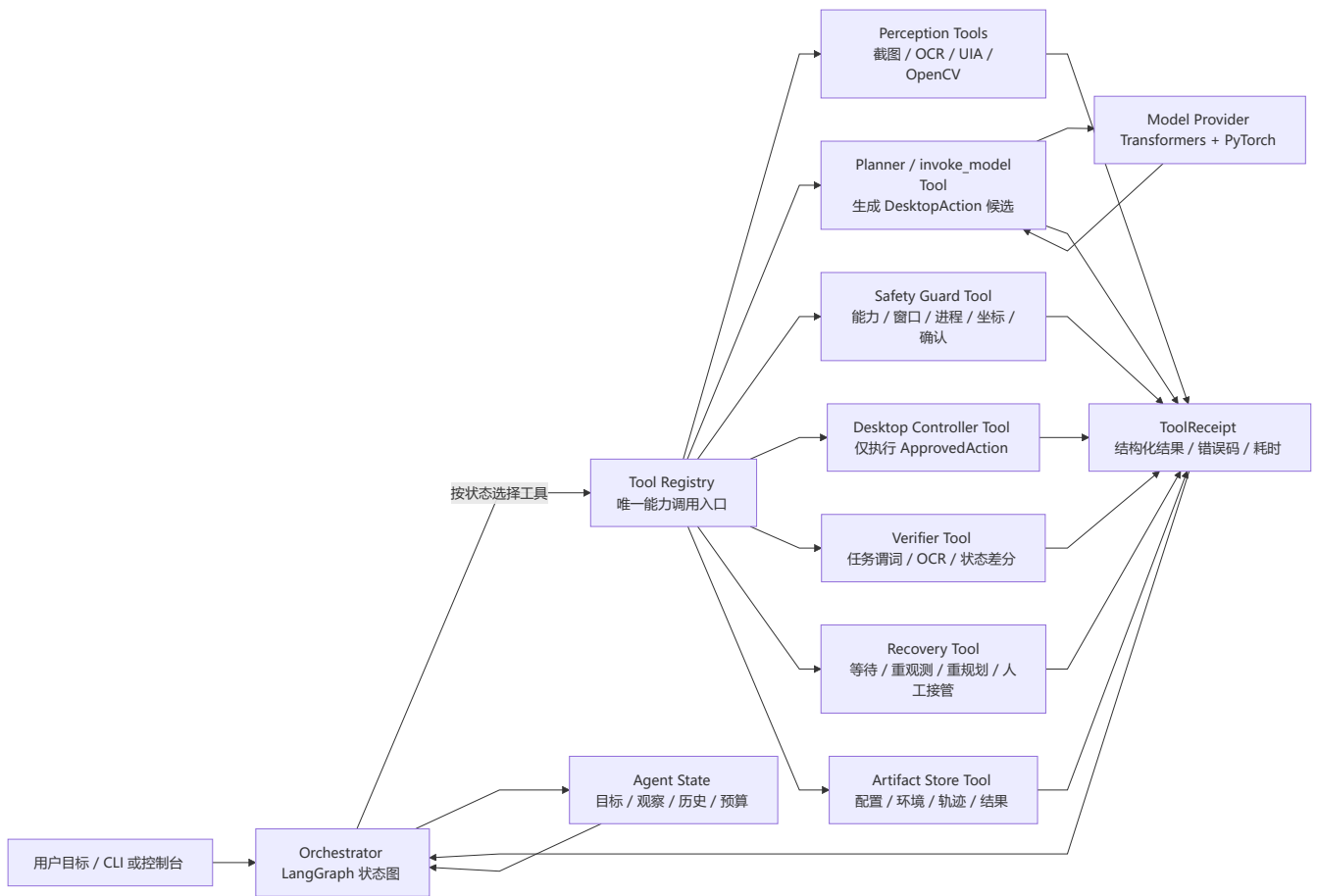
4. 当前实现基线

当前代码位于 `src/max_agent/`，已经实现以下能力：

1. `max-agent doctor`：检查 Python、依赖、CUDA、GPU、BF16，以及 `mss`、PyAutoGUI、OpenCV、PaddleOCR、`pynput` 等基础工具。
2. `--desktop-probe`：显式创建临时 Tk 测试窗口；只有窗口取得前台焦点并通过句柄校验后，才发送受控点击和文本输入。
3. 模型路径约束：模型只能位于仓库根目录 `model/` 树下，避免权重散落到源码或系统缓存目录。
4. 模型管理：通过 ModelScope 显式下载 Qwen3.5-4B，并支持只校验 `config.json` 的元数据检查。
5. 离线基准：通过 Transformers 使用 `local_files_only=True` 加载本地模型，执行单图推理并记录加载耗时、推理耗时、峰值显存和输出。
6. 控制台：Typer/Rich/Prompt Toolkit 是首要 CLI 交互层；Textual 是需显式选择的后续高级 UI。当前普通聊天输入只返回“AI backend is not configured”，不会伪装成已完成推理。
7. 实验归档：每次运行在 `artifacts/` 下写入 `config.yaml`、`environment.json`、`trajectory.jsonl` 和 `result.json`。

当前尚未实现的目标模块包括 `Orchestrator`、`Perception`、`Planner`、`Safety Guard`、`Desktop Controller`、`Verifier`、`Recovery` 和完整的 `Session Safety`。新增感知实现时，应放在 `src/max_agent/tools/perception/`，并通过工具注册表接入编排器；不能创建顶层 `perception` 包或绕过工具入口。

5. 总体架构



编排器是唯一可以推进任务状态的核心组件，`tools/registry.py` 是它调用能力的唯一入口。感知、模型、审批、桌面执行、验证、恢复和归档都通过显式工具契约接入；工具不能自行推进全局循环，不能触发另一个高风险工具，也不能绕过 Guard。图中模块均为目标架构，当前尚未创建空包或占位工具。

6. 模块职责与实现边界

模块	主要输入	主要输出	实现方式与边界
CLI / Console	用户目标、命令参数	任务请求、状态展示、退出码	Typer + Rich + Prompt Toolkit 为首要路径；Textual 为可选适配器；不直接访问鼠标键盘
Orchestrator	Agent State、工具回执	下一状态、轮次控制、终止原因	LangGraph 状态图；不实现 OCR、坐标换算或模型加载
Tool Registry	工具名、结构化输入、Agent State	<code>ToolReceipt</code>	编排器调用能力的唯一入口；统一 schema、权限、超时和可恢复错误码
Perception	屏幕、窗口、显示器信息	Observation、UIElement、状态指纹	<code>mss</code> 、OCR、UIA、OpenCV；只观察，不执行动作
Model Provider	图像、提示词、上下文	结构化模型输出	Transformers/PyTorch 本地推理；不下载模型、不执行桌面动作

模块	主要输入	主要输出	实现方式与边界
Planner	用户目标、 Observation、历史	<code>DesktopAction</code> 候选	只生成单个原子动作，不具有执行权限
Safety Guard	候选动作、窗口状态、策略	<code>ApprovedAction</code> 或拒绝原因	纯规则和运行时状态检查；不以模型自信度代替安全审批
Desktop Controller	<code>ApprovedAction</code>	<code>ExecutionReceipt</code>	仅执行已绑定窗口和物理坐标的动作；不判断任务成功
Verifier	执行前后 Observation、预期谓词	成功/失败证据	OCR、控件状态和图像差分；不自行重试或绕过 Guard
Recovery	失败证据、轮次、预算	等待、重观测、重规划或人工接管	只能提出恢复策略，不能直接执行高风险动作
Session Safety	配置、进程、窗口、能力开关	会话锁、清理状态	管理一次任务的生命周期和输入状态释放
Artifact Store	配置、环境、轨迹、结果	可审计文件集合	使用 JSON/JSONL；不写入密钥、真实个人数据或未经授权截图

7. 核心数据契约

目标闭环采用稳定、可序列化的数据对象。第一版直接使用 Pydantic 定义并校验数据契约，统一约束模型输出、工具输入输出、状态流转和归档记录；禁止以未经校验的自由字典作为模块间接口。

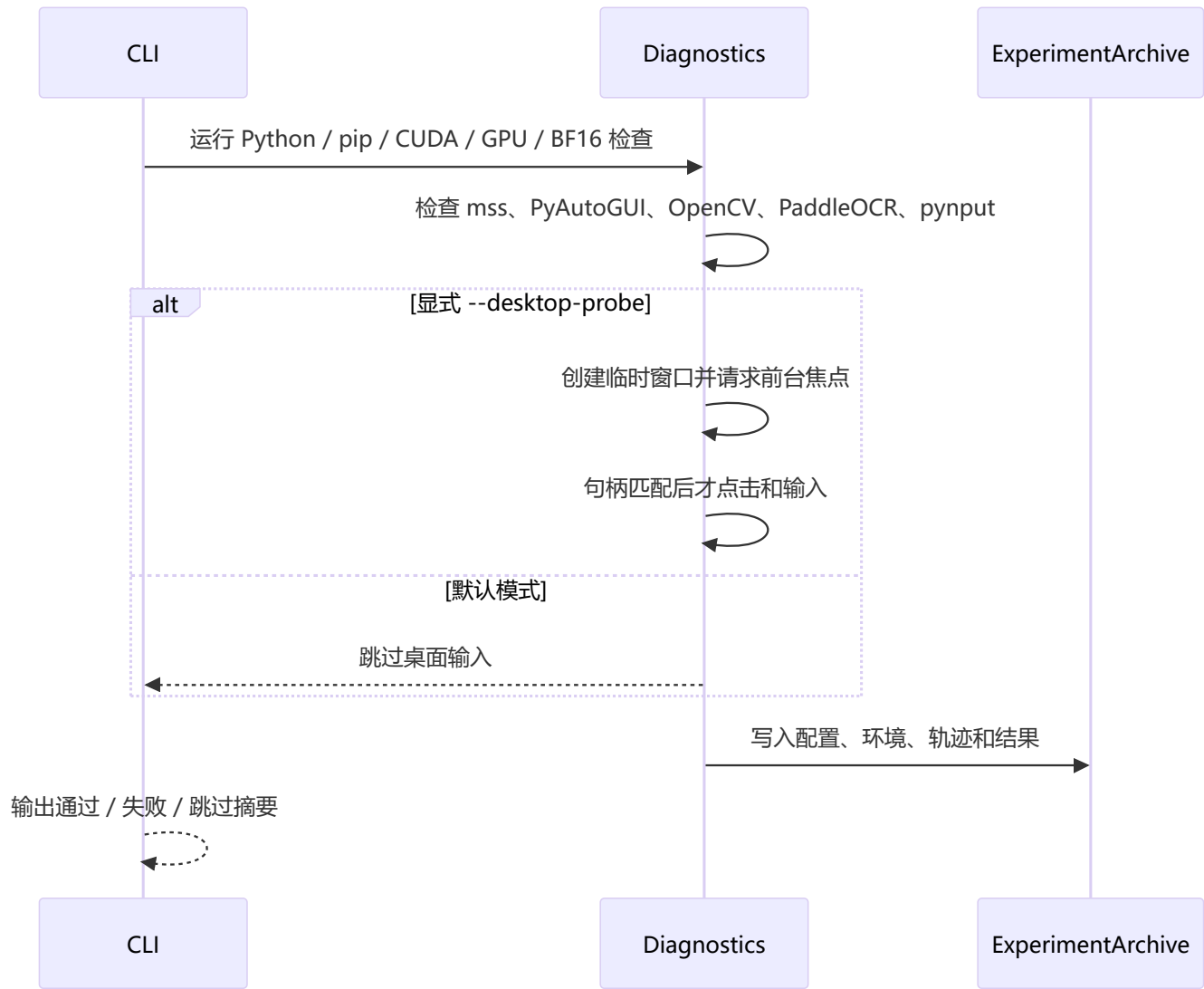
对象	关键字段	语义
<code>ScreenMeta</code>	<code>virtual_rect</code> 、 <code>width_px</code> 、 <code>height_px</code> 、 <code>dpi_scale</code> 、 <code>active_window_rect</code>	描述显示器、虚拟桌面原点和活动窗口
<code>UIElement</code>	<code>text</code> 、 <code>confidence</code> 、 <code>bbox</code> 、 <code>element_id</code> 、 <code>source</code>	来自 UIA、OCR、模板或模型的界面证据
<code>Observation</code>	<code>screenshot_ref</code> 、 <code>screen</code> 、 <code>elements</code> 、 <code>changed_regions</code> 、 <code>state_fingerprint</code>	一次可追溯的屏幕观察
<code>DesktopAction</code>	<code>kind</code> 、目标元素或归一化坐标、文本、预期观察、确认要求	Planner 提出的单个原子动作
<code>ApprovedAction</code>	原动作、窗口句柄、进程 ID、物理坐标、审批哈希	Guard 绑定运行时上下文后生成的唯一可执行动作
<code>ExecutionReceipt</code>	开始/结束时间、前后观察、光标位置、结果、错误码	Controller 对一次动作的执行回执
<code>ToolReceipt</code>	工具名、状态、耗时、结构化输出、可恢复错误码	工具注册表返回编排器的统一回执

对象	关键字段	语义
VerificationResult	任务谓词、证据、成功/失败/不确定、错误码	将结果判定与模型自述分离
RecoveryDecision	等待、重观测、重规划、人工接管、原因和剩余预算	表达有限恢复决定，不直接执行高风险动作
StepRecord	观察、候选动作、审批决定、回执、验证证据	审计单步“为什么做、是否执行、结果如何”

坐标分为两层：感知和模型层使用相对于目标捕获区域的 `[0.0, 1.0]` 归一化坐标；执行层使用 Windows 物理像素。进程在第一次截图或输入前建立 DPI awareness，只有 Guard 可以完成坐标转换和最终越界检查。Pydantic schema 同时负责枚举动作类型、限制坐标范围、校验必填字段、拒绝额外字段，并在工具边界生成可追踪的校验错误。

8. 运行流程与安全策略

8.1 当前 Doctor 流程



8.2 目标 Agent 流程

每轮遵循“一次观察、一个原子动作、一次验证”的最小闭环。Guard 至少检查：

- 总能力开关和单任务会话锁；
- 目标窗口白名单、窗口句柄和进程匹配；
- 归一化坐标转换后的物理像素是否越界；
- 动作间隔、最大轮次、超时和确认策略；
- 输入动作结束后鼠标按键、键盘按键等状态是否已释放。

任何异常都必须停止后续未审批动作，释放输入状态，保留结构化错误和上下文。模型不可用、CUDA/BF16 不可用、本地权重不完整、窗口焦点丢失或验证证据冲突时，系统进入不可执行或人工接管状态，不能降级为无约束控制。

9. 存储、部署与可复现性

- 运行环境：Windows 11、Python `>=3.12,<3.13`、NVIDIA GPU；环境可由 `venv`、`uv` 或 `Conda` 创建，当前实测环境为 `Conda max`。
- 统一安装入口：在仓库根目录运行 `python -m pip install -r requirements.txt`；使用 `uv` 时运行 `uv pip install -r requirements.txt`。根清单包含 `-e .`，会同时安装依赖、当前包和 `max-agent` 命令入口。
- GPU 依赖：根清单显式声明 `PyTorch CUDA 13.0 extra index`，并固定 `torch==2.13.0+cu130` 与匹配的 `TorchVision` 构建。
- 桌面/OCR 依赖：`mss`、`PyAutoGUI`、`OpenCV`、`PaddleOCR`、`PaddlePaddle`、`pynput` 和 `pywinauto` 均由根清单固定版本，不依赖环境中的隐式全局包。
- 模型位置：模型权重只能放在仓库根目录 `model/`；路径由代码校验并使用 `local_files_only=True` 进行离线加载。
- 运行产物：诊断和基准证据写入 `Git` 忽略的 `artifacts/`，至少包括 `config.yaml`、`environment.json`、`trajectory.jsonl`、`result.json`。
- 缓存隔离：`Transformers` 动态模块缓存写入 `artifacts/hf_modules`；模型、缓存、截图和令牌不提交 `Git`。
- 并发策略：第一版单进程、单模型、单并发；不启动 `PostgreSQL`、`Redis` 或消息队列。

当前 `ExperimentArchive` 已能保存部分运行配置、环境摘要、轨迹和最终结果，但 `benchmark` 尚未归档 `max_new_tokens`、不可变模型 `revision/`文件摘要、输入图像摘要、随机种子和完整生成配置，因此当前证据只能支持运行复核，不能保证完整还原一次推理。完整可复现能力的目标契约是：归档上述缺失字段后，同一模型快照、输入摘要、提示词和生成配置能够还原一次基准记录。

10. 分阶段落地路线

阶段	主要实现	验收重点
当前基线	Doctor、控制台壳、模型路径约束、模型下载/校验、离线单图基准、归档	38 项单元测试通过， <code>pip check</code> 通过，命令和现有证据可复核

阶段	主要实现	验收重点
感知与控制	截图、OCR、UIA、窗口元数据、受控点击/输入/滚动/拖拽、急停	只在测试窗口中运行；窗口和坐标校验失败时不输入
基础 Agent	Agent State、Observation/Action schema、Model Provider、Planner	模型只产生结构化候选动作，动作不具备执行权限
端到端闭环	Orchestrator、Guard、Verifier、Recovery、Session Safety	完成若干受控任务，记录每一步状态、证据、失败和退出原因
评测与优化	固定任务集、成功率、步骤数、延迟、显存、失败类型统计	以归档证据和重复实验比较优化，而不是只看模型输出

后续如需 LoRA、更多模型、跨应用任务或服务化部署，应在核心闭环稳定并出现明确指标收益后再引入。所有扩展都不能削弱 Guard、会话锁、离线加载和审计归档这四项基础约束。

11. 当前核验结论

截至 2026-08-07，使用 `F:\Software\Miniconda3\envs\max\python.exe` 对当前工作区执行验证，结果为：

- `python -m pip install -r requirements.txt`：统一清单安装成功，editable 项目入口可用，PyTorch/TorchVision 保持 CUDA 13.0 构建；
- `uv pip install --dry-run --python F:\Software\Miniconda3\envs\max\python.exe -r requirements.txt`：成功解析 121 个包，可消费 extra index 与 editable 项目条目；
- `python -m unittest discover -s tests -v`：38 项测试全部通过；
- `python -m pip check`：No broken requirements found；
- `python -m max_agent.cli --help`：公开命令包含 `doctor`、`download-model`、`validate-model` 和 `benchmark`；
- 非输入 Doctor：Python、依赖、CUDA 13.0、BF16、RTX 5070 Ti、PyAutoGUI、OpenCV、PaddleOCR 和 pynput 检查通过；当前非交互会话的 `mss` 截图因 `BitBlt` 失败，桌面探针按设计跳过，仍需在交互式 Windows 会话复核；
- 已实现的代码边界与本报告的“当前实现基线”一致；
- 感知、规划、审批、执行、验证和恢复仍属于后续设计实现。

12. 结论

MAX 采用“轻量本地运行时 + 显式状态编排 + 能力工具契约 + 独立安全审批 + 证据归档”的架构。当前先把 Windows、CUDA/BF16、本地模型、基础桌面工具和可复现实验打牢，再将这些能力组装为受控 GUI Agent 闭环。

该架构的核心取舍是：用单进程降低原型复杂度，用 LangGraph 保持状态和终止条件显式，用工具边界隔离模型与执行权限，用 Artifact Store 支撑复核。这样既能逐步实现“用模型完成桌面任务”的目标，又不会因为模型输出异常、窗口焦点变化或环境不完整而失去安全控制。